

Usage of AI in the Project

Generative AI: A Tool Worth Using Wisely

The ICA Principle:

1. **Invite** — bring AI in deliberately, as a collaborator, not a default first move.
2. **Control** — you direct it, review everything it produces, and decide what to keep.
3. **Automate** — let it handle the repetitive parts, not the problem-solving itself.

You Are the Problem Solver

AI is how you solve problems effectively — not a replacement for solving them.

- If you just copy an answer without understanding it: you'll know, your teammates will know, and everyone else will eventually know too.
- Be the problem solver AI can't replace.
- Use AI carefully.

AI Use Options

Your options depend on the course level:

- **200-level:** No vibe coding — you must build the code with your own effort. Using AI as support (tests, refactoring, debugging help) is fine.
- **400-level:** Vibe coding is allowed, but you must understand at least 80% of any AI-generated code.
- **Any level:** You can also choose not to use AI at all.

If you decide to use AI at 400-level, the focus shifts from coding to ideas — don't build something AI could produce in 10 minutes.

- Aim higher.
- Solve hard problems.
- Impress others.

What AI Can Help With

Here are the areas where AI can support you as a problem solver:

- Learning programming syntax
- Discussing architecture & design
- Making tests
- Making documentation
- Writing code (if you choose vibe coding)

Learn from AI: Programming & Syntax

Use AI for syntax, not problem-solving.

- Look up unfamiliar syntax or APIs
- Get quick explanations of error messages
- Learn idioms for a new library

Ask "why," not just "what to type."

Discuss Architecture & Design with AI

Use AI as a sounding board for design decisions — not the decision-maker.

- Brainstorm alternative architectures or data models
- Talk through trade-offs before committing
- Get a second opinion on your design

You make the call — AI just helps you think it through faster.

Making Tests

AI is especially good at generating test scaffolding.

- Draft unit tests for a module you've written
- Suggest edge cases you might have missed
- Expand test coverage for existing code

Always read and understand what each test actually checks.

Making Documentation

Let AI handle first drafts, then you edit for accuracy.

- Draft README files, API docs, or code comments
- Summarize what a module does
- Turn your notes into a clean user manual

Documentation from AI still needs your review — verify it matches the actual code.

Writing Code (If You Choose To)

If you've chosen vibe coding, AI can also write the code itself — under your direction.

- Describe the feature or module you want
- Review and test what it produces before accepting it
- You're still responsible for understanding and defending it

Required Safeguards and Disclosure

Regardless of the level you choose:

- Follow this course's AI policy
- Disclose meaningful AI use — tool, purpose, and key generated artifacts
- Verify AI output with tests and review
- Never upload secrets, personal data, or restricted code

You're Still Accountable

- Explain the architecture and behavior of what you submit, and justify important implementation decisions
- Be able to debug and modify your submitted code independently
- AI mistakes are your mistakes — you own the final result

- If you used vibe coding (400-level), the 80% understanding bar isn't optional — you may be asked to explain any part of the AI-generated code

AI use is optional. If you use it, using it responsibly — not just using it — is part of the evaluation.